

# DAY 2: Instructor module, BACKEND

Registration, admin approval or rejection (with a reason), application status, and course CRUD for approved instructors. **10 new files + 1 edit.** The `Course` table and the existing `/api/course` API are untouched.

**Needs Day 1 first** (the instructor registration uses `OtpService`).

## 1. The idea

| Piece                 | Where it lives                                                                                                                                                   |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| the login account     | an ordinary row in <code>users</code> with role <b>INSTRUCTOR</b> (so the JWT, refresh cookie and login need no change)                                          |
| the application       | <code>instructor_profiles</code> : qualification, years of experience, expertise, bio, optional link, <b>status</b> and the rejection reason                     |
| who owns which course | <code>course_owners</code> (course id plus instructor id). A separate table, so <code>Course</code> is not changed. Courses with no row here belong to the admin |

### Status life cycle

```
register -> PENDING --admin approves--> APPROVED
          |
          +--admin rejects (reason 5-500 chars)--> REJECTED --instructor edits and resubmits-->
PENDING
```

- An instructor **can log in while PENDING or REJECTED**. That is how they see progress and the rejection reason on the registration page.
- Only **APPROVED** instructors can use course CRUD. Others get 403 "Your instructor account is not approved yet".
- Rejecting needs a reason. Approving a REJECTED application is allowed (admin changes their mind).
- An approved application cannot be edited.

## 2. Endpoints

| Method and path                                      | Who                             | What                                                         |
|------------------------------------------------------|---------------------------------|--------------------------------------------------------------|
| POST <code>/api/instructor/register</code>           | public (10 per hour per IP)     | creates the account and the PENDING application              |
| GET <code>/api/instructor/me</code>                  | INSTRUCTOR                      | my application and status                                    |
| PUT <code>/api/instructor/me</code>                  | INSTRUCTOR                      | fix details (PENDING) or resubmit (REJECTED becomes PENDING) |
| GET/POST <code>/api/instructor/courses</code>        | approved INSTRUCTOR             | my courses, add a course                                     |
| PUT/DELETE <code>/api/instructor/courses/{id}</code> | approved INSTRUCTOR who owns it | edit, delete                                                 |

| Method and path                              | Who   | What                                         |
|----------------------------------------------|-------|----------------------------------------------|
| GET /api/admin/instructors?status=           | ADMIN | list ( PENDING , APPROVED , REJECTED , ALL ) |
| POST /api/admin/instructors/{userId}/approve | ADMIN | approve                                      |
| POST /api/admin/instructors/{userId}/reject  | ADMIN | body { "reason": "..."}                      |

Course CRUD calls your existing `CourseService`, so all course rules still apply. "Not yours" and "does not exist" give the same answer so ids cannot be probed.

`CourseDeleteListener` is a small hook: anything that owns data tied to a course (the Day 3 videos) implements it and is told when an instructor deletes the course. It is looked up only at delete time so there is no start-up dependency loop.

### 3. Honest status

Compiled with real Java against stand-in Spring classes and run with fake repositories: **49 checks, 0 failures** (registration rules, every status change, rejection reason rules, ownership, not-approved blocking, course delete hook). The browser side passed **31 checks** (see the frontend guide). **Not tested:** Spring start-up, MySQL, real HTTP security. Send me any compiler message.

**Known limit:** if the **admin** deletes an instructor's course through the existing admin screens, the ownership row stays behind (harmless) and Day 3 videos of that course stay on disk. Deleting from the instructor's own page cleans up fully.

### 4. The one edit: `config/SecurityConfig.java`

Add these two lines **above** `.anyRequest().authenticated()` (and above any rule that would match them first). The register line must come first:

```
.requestMatchers("/api/instructor/register").permitAll()
.requestMatchers("/api/instructor/**").hasAuthority("INSTRUCTOR")
```

`/api/admin/**` is already ADMIN-only, so the admin instructor endpoints need no new rule.

### 5. Check it

1. Register: POST `/api/instructor/register` with username, email, password, mobileNumber, qualification, experienceYears, expertise, bio (20+ characters), profileLink (optional). Answer 201 with `status: PENDING`.
2. Log in as that instructor, GET `/api/instructor/me`: PENDING.
3. POST `/api/instructor/courses` now: **403 not approved**.
4. As admin: GET `/api/admin/instructors?status=PENDING`, then POST `.../{userId}/reject` with `{"reason":"abc"}` (too short: 400), then with a real reason (200).
5. As the instructor: `/me` shows `rejectionReason`. PUT `/me` with fixed details: PENDING again.

6. Admin approves. The instructor adds, edits and deletes a course. Another instructor cannot touch it.

## 6. Troubleshooting

| Symptom                                          | Cause                                                                                                                    |
|--------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| 401/403 on register                              | the <code>permitAll</code> line is below the <code>hasAuthority</code> line                                              |
| instructor gets 403 on every call                | the JWT role is not <code>INSTRUCTOR</code> ; check the <code>users.role</code> value                                    |
| <code>NoSuchBeanDefinition ... OtpService</code> | Day 1 files are missing                                                                                                  |
| circular dependency error at start-up            | an older copy of <code>InstructorCourseService</code> that read the listener list in its constructor; use the file below |

## 7. Complete code

**model/InstructorProfile.java** (new file)

```
package com.examly.springapp.model;

import jakarta.persistence.*;
import java.time.LocalDateTime;

/**
 * The application of an instructor: what they told us about themselves and where the review stands.
 * status is PENDING, APPROVED or REJECTED; a rejection always carries a reason for the instructor to read.
 * The login account itself is an ordinary row in users with role INSTRUCTOR.
 */
@Entity
@Table(name = "instructor_profiles")
public class InstructorProfile {
    public static final String PENDING = "PENDING";
    public static final String APPROVED = "APPROVED";
    public static final String REJECTED = "REJECTED";

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long profileId;

    @Column(nullable = false, unique = true)
    private Long userId;

    @Column(length = 100)
    private String qualification;

    private int experienceYears;

    @Column(length = 150)
    private String expertise;

    @Column(length = 1000)
    private String bio;

    @Column(length = 300)
    private String profileLink;

    @Column(length = 20, nullable = false)
    private String status = PENDING;

    @Column(length = 500)
    private String rejectionReason;

    private LocalDateTime submittedAt;
```

```

private LocalDateTime reviewedAt;

@Column(length = 120)
private String reviewedBy;

public InstructorProfile() {}

public Long getProfileId() { return profileId; }
public Long getUserId() { return userId; }
public void setUserId(Long userId) { this.userId = userId; }
public String getQualification() { return qualification; }
public void setQualification(String qualification) { this.qualification = qualification; }
public int getExperienceYears() { return experienceYears; }
public void setExperienceYears(int experienceYears) { this.experienceYears = experienceYears; }
public String getExpertise() { return expertise; }
public void setExpertise(String expertise) { this.expertise = expertise; }
public String getBio() { return bio; }
public void setBio(String bio) { this.bio = bio; }
public String getProfileLink() { return profileLink; }
public void setProfileLink(String profileLink) { this.profileLink = profileLink; }
public String getStatus() { return status; }
public void setStatus(String status) { this.status = status; }
public String getRejectionReason() { return rejectionReason; }
public void setRejectionReason(String rejectionReason) { this.rejectionReason = rejectionReason; }
public LocalDateTime getSubmittedAt() { return submittedAt; }
public void setSubmittedAt(LocalDateTime submittedAt) { this.submittedAt = submittedAt; }
public LocalDateTime getReviewedAt() { return reviewedAt; }
public void setReviewedAt(LocalDateTime reviewedAt) { this.reviewedAt = reviewedAt; }
public String getReviewedBy() { return reviewedBy; }
public void setReviewedBy(String reviewedBy) { this.reviewedBy = reviewedBy; }
}

```

#### model/CourseOwner.java (new file)

```

package com.examly.springapp.model;

import jakarta.persistence.*;

/**
 * Says which instructor created a course. It is a separate table so the Course table (and the course API
 * the tests rely on) stays exactly as it was. Courses without a row here were made by an admin.
 */
@Entity
@Table(name = "course_owners")
public class CourseOwner {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private Long courseId;

    @Column(nullable = false)
    private Long instructorUserId;

    public CourseOwner() {}

    public CourseOwner(Long courseId, Long instructorUserId) {
        this.courseId = courseId;
        this.instructorUserId = instructorUserId;
    }

    public Long getId() { return id; }
    public Long getCourseId() { return courseId; }
    public Long getInstructorUserId() { return instructorUserId; }
}

```

#### repository/InstructorProfileRepo.java (new file)

```

package com.examly.springapp.repository;

import com.examly.springapp.model.InstructorProfile;
import java.util.List;
import java.util.Optional;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

/** Data access for instructor applications. */
@Repository
public interface InstructorProfileRepo extends JpaRepository<InstructorProfile, Long> {
    Optional<InstructorProfile> findById(Long userId);
    List<InstructorProfile> findByStatusOrderBySubmittedAtDesc(String status);
    List<InstructorProfile> findAllByOrderBySubmittedAtDesc();
}

```

### repository/CourseOwnerRepo.java (new file)

```

package com.examly.springapp.repository;

import com.examly.springapp.model.CourseOwner;
import java.util.List;
import java.util.Optional;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

/** Data access for course ownership. */
@Repository
public interface CourseOwnerRepo extends JpaRepository<CourseOwner, Long> {
    Optional<CourseOwner> findById(Long courseId);
    List<CourseOwner> findByInstructorUserId(Long instructorUserId);
    void deleteByCourseId(Long courseId);
}

```

### dto/InstructorDtos.java (new file)

```

package com.examly.springapp.dto;

import java.time.LocalDateTime;

/** Request and response shapes of the instructor API. */
public final class InstructorDtos {
    private InstructorDtos() {}

    /** Public instructor registration: the account fields plus the application details. */
    public record InstructorRegisterRequest(String email, String password, String username, String
mobileNumber,
                                           String qualification, Integer experienceYears, String
expertise,
                                           String bio, String profileLink) {}

    /** The editable application details (used to fix a pending or rejected application). */
    public record InstructorDetails(String qualification, Integer experienceYears, String expertise,
                                   String bio, String profileLink) {}

    public record RejectRequest(String reason) {}

    /** What the instructor and the admin see. The password never appears here. */
    public record InstructorView(Long userId, String username, String email, String mobileNumber,
                                String qualification, int experienceYears, String expertise, String bio,
                                String profileLink, String status, String rejectionReason,
                                LocalDateTime submittedAt, LocalDateTime reviewedAt, String reviewedBy) {}
}

```

### service/CourseDeleteListener.java (new file)

```

package com.examly.springapp.service;

/**
 * Anything that owns data tied to a course (for example its videos) can implement this so that data is
 * cleaned up when an instructor deletes the course.
 */
public interface CourseDeleteListener {
    void onCourseDeleted(Long courseId);
}

```

### service/InstructorService.java (new file)

```

package com.examly.springapp.service;

import com.examly.springapp.dto.InstructorDtos.*;
import com.examly.springapp.exception.DatabaseOperationException;
import com.examly.springapp.exception DuplicateResourceException;
import com.examly.springapp.exception.InvalidRequestException;
import com.examly.springapp.exception.ResourceNotFoundException;
import com.examly.springapp.model.InstructorProfile;
import com.examly.springapp.model.User;
import com.examly.springapp.repository.InstructorProfileRepo;
import com.examly.springapp.repository.UserRepo;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;
import java.util.Locale;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.dao.DataIntegrityViolationException;
import org.springframework.security.access.AccessDeniedException;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;

/**
 * Instructor registration, the admin's approve / reject decision (a rejection needs a reason) and the
 * "is this instructor approved?" check used before an instructor may manage courses or videos.
 */
@Service
public class InstructorService {
    private static final Logger log = LoggerFactory.getLogger(InstructorService.class);
    private static final String PASSWORD_POLICY = "^(?=.*[A-Za-z])(?=.*\\d){8,72}$";
    static final int MIN_REASON = 5, MAX_REASON = 500;

    private final InstructorProfileRepo profiles;
    private final UserRepo users;
    private final PasswordEncoder encoder;
    private final OtpService otp;

    @Autowired
    public InstructorService(InstructorProfileRepo profiles, UserRepo users, PasswordEncoder encoder,
        OtpService otp) {
        this.profiles = profiles;
        this.users = users;
        this.encoder = encoder;
        this.otp = otp;
    }

    // ----- registration

    public InstructorView register(InstructorRegisterRequest req) {
        if (req == null) throw new InvalidRequestException("Registration details are required");
        String email = req.email() == null ? "" : req.email().trim().toLowerCase(Locale.ROOT);
        if (!email.matches("^[^@\\s]+@[^@\\s]+\\.([^@\\s]+$") || email.length() > 120) {
            throw new InvalidRequestException("Email must be valid");
        }
        String username = req.username() == null ? "" : req.username().trim();

```

```

        if (username.isEmpty() || username.length() > 50 || username.contains(" ")) {
            throw new InvalidRequestException("Username is required, without spaces, up to 50 characters");
        }
        if (req.password() == null || !req.password().matches(PASSWORD_POLICY)) {
            throw new InvalidRequestException("Password must be 8 to 72 characters and include a letter and
a digit");
        }
        if (req.mobileNumber() == null || !req.mobileNumber().matches("[0-9]{10}$")) {
            throw new InvalidRequestException("Mobile number must be 10 digits");
        }
        InstructorDetails details = checkDetails(new InstructorDetails(req.qualification(),
req.experienceYears(),
        req.expertise(), req.bio(), req.profileLink()));
        if (DatabaseOperationException.guard("checking the email", () -> users.existsByEmail(email))) {
            throw new DuplicateResourceException("A user with this email already exists");
        }
        User user = new User();
        user.setEmail(email);
        user.setUsername(username);
        user.setMobileNumber(req.mobileNumber());
        user.setRole("INSTRUCTOR");
        user.setPassword(encoder.encode(req.password()));
        User saved;
        try {
            saved = users.save(user);
        } catch (DataIntegrityViolationException e) {
            throw new DuplicateResourceException("A user with this email already exists");
        }
        InstructorProfile p = new InstructorProfile();
        p.setUserId(saved.getUserId());
        apply(p, details);
        p.setStatus(InstructorProfile.PENDING);
        p.setSubmittedAt(LocalDateDateTime.now());
        DatabaseOperationException.guardVoid("saving the application", () -> profiles.save(p));
        otp.markPending(saved);
        log.info("Instructor application {} submitted", saved.getUserId());
        return view(saved, p);
    }

    // ----- the instructor's own application

    public InstructorView myProfile(String email) {
        User u = userByEmail(email);
        return view(u, profileOf(u.getUserId()));
    }

    /** Fixing the application: allowed while PENDING, and a REJECTED application goes back to PENDING. */
    public InstructorView updateMine(String email, InstructorDetails in) {
        User u = userByEmail(email);
        InstructorProfile p = profileOf(u.getUserId());
        if (InstructorProfile.APPROVED.equals(p.getStatus())) {
            throw new InvalidRequestException("An approved application can no longer be edited here");
        }
        apply(p, checkDetails(in));
        if (InstructorProfile.REJECTED.equals(p.getStatus())) {
            p.setStatus(InstructorProfile.PENDING);
            p.setRejectionReason(null);
            p.setSubmittedAt(LocalDateDateTime.now());
            p.setReviewedAt(null);
            p.setReviewedBy(null);
        }
        DatabaseOperationException.guardVoid("saving the application", () -> profiles.save(p));
        return view(u, p);
    }

    /** Used before course and video management: only an APPROVED instructor may continue. */
    public User requireApproved(String email) {
        User u = userByEmail(email);
        if (!"INSTRUCTOR".equals(u.getRole())) throw new AccessDeniedException("Instructors only");
        InstructorProfile p = profileOf(u.getUserId());
        if (!InstructorProfile.APPROVED.equals(p.getStatus())) {

```

```

        throw new AccessDeniedException("Your instructor account is not approved yet");
    }
    return u;
}

// ----- the admin's review

public List<InstructorView> list(String status) {
    List<InstructorProfile> rows;
    if (status == null || status.isBlank() || status.equalsIgnoreCase("ALL")) {
        rows = DatabaseOperationException.guard("listing applications",
profiles::findAllByOrderBySubmittedAtDesc);
    } else {
        String s = status.trim().toUpperCase(Locale.ROOT);
        if (!List.of("PENDING", "APPROVED", "REJECTED").contains(s)) {
            throw new InvalidRequestException("Status must be PENDING, APPROVED, REJECTED or ALL");
        }
        rows = DatabaseOperationException.guard("listing applications", () ->
profiles.findByStatusOrderBySubmittedAtDesc(s));
    }
    List<InstructorView> out = new ArrayList<>();
    for (InstructorProfile p : rows) {
        User u = DatabaseOperationException.guard("loading a user", () ->
users.findById(p.getUserId()).orElse(null);
        if (u != null) out.add(view(u, p));
    }
    return out;
}

public InstructorView approve(Long userId, String adminEmail) {
    User u = userById(userId);
    InstructorProfile p = profileOf(userId);
    if (InstructorProfile.APPROVED.equals(p.getStatus())) throw new InvalidRequestException("This
instructor is already approved");
    p.setStatus(InstructorProfile.APPROVED);
    p.setRejectionReason(null);
    p.setReviewedAt(LocalDate.now());
    p.setReviewedBy(adminEmail);
    DatabaseOperationException.guardVoid("saving the decision", () -> profiles.save(p));
    log.info("Instructor {} approved by {}", userId, adminEmail);
    return view(u, p);
}

public InstructorView reject(Long userId, String reasonRaw, String adminEmail) {
    String reason = reasonRaw == null ? "" : reasonRaw.replaceAll("\\p{Cntrl}", " ").trim();
    if (reason.length() < MIN_REASON || reason.length() > MAX_REASON) {
        throw new InvalidRequestException("Give a reason of " + MIN_REASON + " to " + MAX_REASON + "
characters");
    }
    User u = userById(userId);
    InstructorProfile p = profileOf(userId);
    if (!InstructorProfile.PENDING.equals(p.getStatus())) {
        throw new InvalidRequestException("Only a pending application can be rejected");
    }
    p.setStatus(InstructorProfile.REJECTED);
    p.setRejectionReason(reason);
    p.setReviewedAt(LocalDate.now());
    p.setReviewedBy(adminEmail);
    DatabaseOperationException.guardVoid("saving the decision", () -> profiles.save(p));
    log.info("Instructor {} rejected by {}", userId, adminEmail);
    return view(u, p);
}

// ----- helpers

private User userByEmail(String email) {
    return DatabaseOperationException.guard("loading a user", () -> users.findByEmail(email == null ?
"" : email.trim().toLowerCase(Locale.ROOT)))
        .orElseThrow(() -> new ResourceNotFoundException("User not found"));
}

```



```

private User userById(Long id) {
    return DatabaseOperationException.guard("loading a user", () -> users.findById(id))
        .filter(x -> "INSTRUCTOR".equals(x.getRole()))
        .orElseThrow(() -> new ResourceNotFoundException("Instructor not found"));
}

private InstructorProfile profileOf(Long userId) {
    return DatabaseOperationException.guard("loading the application", () ->
profiles.findById(userId))
        .orElseThrow(() -> new ResourceNotFoundException("Instructor application not found"));
}

private static void apply(InstructorProfile p, InstructorDetails d) {
    p.setQualification(d.qualification());
    p.setExperienceYears(d.experienceYears());
    p.setExpertise(d.expertise());
    p.setBio(d.bio());
    p.setProfileLink(d.profileLink());
}

/** Checks and trims the application fields. Returns a clean copy. */
static InstructorDetails checkDetails(InstructorDetails d) {
    if (d == null) throw new InvalidRequestException("Application details are required");
    String q = clean(d.qualification());
    String e = clean(d.expertise());
    String b = clean(d.bio());
    String link = clean(d.profileLink());
    if (q.length() < 2 || q.length() > 100) throw new InvalidRequestException("Qualification is
required (2 to 100 characters)");
    if (d.experienceYears() == null || d.experienceYears() < 0 || d.experienceYears() > 60) {
        throw new InvalidRequestException("Years of experience must be between 0 and 60");
    }
    if (e.length() < 2 || e.length() > 150) throw new InvalidRequestException("Area of expertise is
required (2 to 150 characters)");
    if (b.length() < 20 || b.length() > 1000) throw new InvalidRequestException("Short bio must be 20
to 1000 characters");
    if (!link.isEmpty() && (!link.matches("^https?://\\S+$") || link.length() > 300)) {
        throw new InvalidRequestException("Profile link must start with http:// or https://");
    }
    return new InstructorDetails(q, d.experienceYears(), e, b, link);
}

private static String clean(String s) {
    return s == null ? "" : s.replaceAll("\\p{Cntrl}", " ").trim();
}

static InstructorView view(User u, InstructorProfile p) {
    return new InstructorView(u.getUserId(), u.getUsername(), u.getEmail(), u.getMobileNumber(),
p.getQualification(),
        p.getExperienceYears(), p.getExpertise(), p.getBio(), p.getProfileLink(), p.getStatus(),
p.getRejectionReason(), p.getSubmittedAt(), p.getReviewedAt(), p.getReviewedBy());
}
}

```

## service/InstructorCourseService.java (new file)

```

package com.examly.springapp.service;

import com.examly.springapp.exception.DatabaseOperationException;
import com.examly.springapp.exception.ResourceNotFoundException;
import com.examly.springapp.model.Course;
import com.examly.springapp.model.CourseOwner;
import com.examly.springapp.model.User;
import com.examly.springapp.repository.CourseOwnerRepo;
import com.examly.springapp.repository.CourseRepo;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;
import org.springframework.beans.factory.ObjectProvider;

```

```

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.security.access.AccessDeniedException;
import org.springframework.stereotype.Service;

/**
 * Course management for an APPROVED instructor: create, view, edit and delete, always only their own
 * courses.
 * The real work is done by the existing CourseService, so every course rule stays in one place.
 */
@Service
public class InstructorCourseService {
    private final InstructorService instructors;
    private final CourseService courseService;
    private final CourseRepo courseRepo;
    private final CourseOwnerRepo owners;
    // looked up only when a course is deleted, so VideoService (which needs this class) can be one of them
    without a start-up loop
    private final ObjectProvider<CourseDeleteListener> listeners;

    @Autowired
    public InstructorCourseService(InstructorService instructors, CourseService courseService, CourseRepo
courseRepo,
                                CourseOwnerRepo owners,
                                ObjectProvider<CourseDeleteListener> listeners) {
        this.instructors = instructors;
        this.courseService = courseService;
        this.courseRepo = courseRepo;
        this.owners = owners;
        this.listeners = listeners;
    }

    public Course create(String email, Course course) {
        User u = instructors.requireApproved(email);
        Course saved = courseService.addCourse(course);
        DatabaseOperationException.guardVoid("saving the course owner", () -> owners.save(new
CourseOwner(saved.getCourseId(), u.getUserId())));
        return saved;
    }

    public List<Course> mine(String email) {
        User u = instructors.requireApproved(email);
        List<CourseOwner> rows = DatabaseOperationException.guard("loading your courses", () ->
owners.findByInstructorUserId(u.getUserId()));
        List<Course> out = new ArrayList<>();
        for (CourseOwner o : rows) {
            Optional<Course> c = DatabaseOperationException.guard("loading a course", () ->
courseRepo.findById(o.getCourseId()));
            c.ifPresent(out::add); // a course deleted by an admin simply disappears from the list
        }
        return out;
    }

    public Course update(String email, Long courseId, Course course) {
        User u = instructors.requireApproved(email);
        requireOwner(u, courseId);
        return courseService.updateCourse(courseId, course);
    }

    public void delete(String email, Long courseId) {
        User u = instructors.requireApproved(email);
        requireOwner(u, courseId);
        listeners.orderedStream().forEach(l -> l.onCourseDeleted(courseId));
        courseService.deleteCourse(courseId);
        DatabaseOperationException.guardVoid("removing the course owner", () ->
owners.deleteByCourseId(courseId));
    }

    /** True when this approved instructor owns the course. */
    public boolean owns(User instructor, Long courseId) {
        Optional<CourseOwner> o = DatabaseOperationException.guard("checking the course owner", () ->
owners.findByCourseId(courseId));
    }

```

```

        return o.isPresent() && o.get().getInstructorUserId().equals(instructor.getUserId());
    }

    private void requireOwner(User u, Long courseId) {
        if (courseId == null || courseId <= 0) throw new ResourceNotFoundException("Course not found");
        if (!owns(u, courseId)) {
            // the same answer for "not yours" and "does not exist", so ids cannot be probed
            throw new AccessDeniedException("You can only manage your own courses");
        }
    }
}

```

## controller/InstructorController.java (new file)

```

package com.examly.springapp.controller;

import com.examly.springapp.config.RateLimiter;
import com.examly.springapp.dto.InstructorDtos.*;
import com.examly.springapp.exception.RateLimitExceededException;
import com.examly.springapp.model.Course;
import com.examly.springapp.service.AccessService;
import com.examly.springapp.service.InstructorCourseService;
import com.examly.springapp.service.InstructorService;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.validation.Valid;
import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

/** Instructor registration (public), the instructor's own application, and their course CRUD (approved only). */
@RestController
@RequestMapping("/api/instructor")
@Tag(name = "Instructor", description = "Instructor registration, application status and own courses")
public class InstructorController {
    private final InstructorService instructors;
    private final InstructorCourseService courses;
    private final AccessService access;
    private final RateLimiter limiter;

    @Autowired
    public InstructorController(InstructorService instructors, InstructorCourseService courses,
        AccessService access, RateLimiter limiter) {
        this.instructors = instructors;
        this.courses = courses;
        this.access = access;
        this.limiter = limiter;
    }

    @Operation(summary = "Register as an instructor (public)",
        description = "Creates the account (role INSTRUCTOR) and a PENDING application. The admin decides later.")
    @PostMapping("/register")
    public ResponseEntity<InstructorView> register(@RequestBody InstructorRegisterRequest body,
        HttpServletRequest request) {
        String key = "instructor-register:" + request.getRemoteAddr();
        if (!limiter.tryAcquire(key, 10, 60 * 60 * 1000L)) {
            throw new RateLimitExceededException("Too many registrations from this device. Try again later.", limiter.retryAfterSeconds(key, 60 * 60 * 1000L));
        }
        return ResponseEntity.status(HttpStatus.CREATED).body(instructors.register(body));
    }

    @Operation(summary = "My application and its status")
    @GetMapping("/me")

```

```

    public ResponseEntity<InstructorView> me() {
        return ResponseEntity.ok(instructors.myProfile(access.currentEmail()));
    }

    @Operation(summary = "Fix my application", description = "Allowed while PENDING; a REJECTED application goes back to PENDING.")
    @PutMapping("/me")
    public ResponseEntity<InstructorView> updateMe(@RequestBody InstructorDetails body) {
        return ResponseEntity.ok(instructors.updateMine(access.currentEmail(), body));
    }

    @Operation(summary = "My courses (approved instructors)")
    @GetMapping("/courses")
    public ResponseEntity<List<Course>> myCourses() {
        return ResponseEntity.ok(courses.mine(access.currentEmail()));
    }

    @Operation(summary = "Add a course (approved instructors)")
    @PostMapping("/courses")
    public ResponseEntity<Course> add(@Valid @RequestBody Course course) {
        return ResponseEntity.status(HttpStatus.CREATED).body(courses.create(access.currentEmail(), course));
    }

    @Operation(summary = "Edit one of my courses")
    @PutMapping("/courses/{courseId}")
    public ResponseEntity<Course> update(@PathVariable Long courseId, @Valid @RequestBody Course course) {
        return ResponseEntity.ok(courses.update(access.currentEmail(), courseId, course));
    }

    @Operation(summary = "Delete one of my courses")
    @DeleteMapping("/courses/{courseId}")
    public ResponseEntity<Void> delete(@PathVariable Long courseId) {
        courses.delete(access.currentEmail(), courseId);
        return ResponseEntity.noContent().build();
    }
}

```

### controller/AdminInstructorController.java (new file)

```

package com.examly.springapp.controller;

import com.examly.springapp.dto.InstructorDtos.*;
import com.examly.springapp.service.AccessService;
import com.examly.springapp.service.InstructorService;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.tags.Tag;
import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

/** The admin's view of instructor applications (the /api/admin/** rule already limits this to ADMIN). */
@RestController
@RequestMapping("/api/admin/instructors")
@Tag(name = "Instructor approvals", description = "Admin only: list, approve or reject instructor applications")
public class AdminInstructorController {
    private final InstructorService instructors;
    private final AccessService access;

    @Autowired
    public AdminInstructorController(InstructorService instructors, AccessService access) {
        this.instructors = instructors;
        this.access = access;
    }

    @Operation(summary = "List applications", description = "Optional ? status=PENDING|APPROVED|REJECTED|ALL")

```

```

@GetMapping
public ResponseEntity<List<InstructorView>> list(@RequestParam(required = false) String status) {
    return ResponseEntity.ok(instructors.list(status));
}

@Operation(summary = "Approve an instructor")
@PostMapping("/{userId}/approve")
public ResponseEntity<InstructorView> approve(@PathVariable Long userId) {
    return ResponseEntity.ok(instructors.approve(userId, access.currentEmail()));
}

@Operation(summary = "Reject an instructor", description = "Body: {reason} (5 to 500 characters, shown to the instructor).")
@PostMapping("/{userId}/reject")
public ResponseEntity<InstructorView> reject(@PathVariable Long userId, @RequestBody RejectRequest body) {
    return ResponseEntity.ok(instructors.reject(userId, body == null ? null : body.reason(), access.currentEmail()));
}
}

```